# ARCUS-H: Behavioral Stability Under Controlled Stress as a Complementary RL Evaluation Axis

Karim ZINEBI
Independent Researcher
karim.zinebiof@gmail.com

March 17, 2026

## Abstract

Reinforcement learning agents are typically evaluated by expected episodic return, yet return can hide fragility: policies that appear strong in nominal conditions may degrade when constraints tighten, action semantics become unreliable, or reward feedback is adversarially inverted. We present ARCUS-H (Adaptive Reinforcement Coherence Under Stress Harness), an evaluation harness that measures *behavioral stability* under stress using five interpretable channels—competence, coherence, continuity, integrity, and meaning—combined into a composite stability score. A key calibration contribution is an adaptive per-run threshold derived from the pre-phase score distribution, which achieves a false positive rate of $2.0\%$ (target $\alpha = 0.05$) without any environment-specific tuning.

We benchmark 7 algorithms across 9 environments (6 classic control, 2 MuJoCo, 1 Atari) under 4 stress schedules (concept drift, resource constraint, trust violation, valence inversion) with 10 seeds each. Three findings stand out. First, reward and stability diverge substantially: Pearson $r = +0.14$, $p = 0.364$ between normalized return and collapse rate under valence inversion, confirming that return alone does not capture stress fragility. Second, stressor effects are environment-class-dependent: MuJoCo agents collapse at rates of $66$–$84\%$ compared to $33$–$69\%$ for classic control under the same stressors, despite MuJoCo agents achieving higher absolute reward. Third, each stressor leaves a distinct signature across stability channels, supporting the interpretability of per-channel diagnostics. Evaluation mode robustness is strong: deterministic and stochastic verdicts correlate at $r = 0.82$–$0.96$.

## 1 Introduction

Benchmarking in reinforcement learning typically reduces an evaluation run to a single scalar: episodic return. Return is necessary for measuring task performance, but does not capture whether behavior remains stable when evaluation conditions deviate from training assumptions. In deployed settings, agents may face reduced control authority, mismatches between intended and executed actions, or reward feedback that becomes unreliable or adversarially corrupted. Under such stress, two agents with identical return can exhibit very different behavioral stability profiles.

ARCUS-H addresses this gap by applying controlled stress schedules and computing stability metrics from a behavioral stability signal constructed from five per-episode channels. The goal is not to replace return, but to provide a complementary evaluation axis that surfaces collapse under stress in a reproducible, algorithm-agnostic way. Crucially, ARCUS-H requires no changes to training: it is an evaluation-time harness applied to already-trained policies.

**Contributions.**

1. A phase-based stress evaluation protocol (pre/shock/post) with four ecologically motivated stressors: concept drift, resource constraint, trust violation, and valence inversion.

2. An adaptive calibration procedure (per-run p95 threshold) that achieves target FPR of $\approx \alpha$ without environment-specific tuning.

3. Formal stability metrics derived from five behavioral channels, with direct robust z-scoring that avoids the degenerate MAD-zero failure mode of deficit-based scoring.

4. Empirical results across 9 environments, 7 algorithms, and 10 seeds demonstrating reward/stability divergence, stressor-specific component signatures, and environment-class-dependent vulnerability patterns.

## 2 Related Work

Reproducibility and evaluation reliability in deep RL have received sustained attention. Henderson et al. [1] demonstrate that results are highly sensitive to random seeds and hyperparameters. Agarwal et al. [2] propose stratified bootstrap confidence intervals and aggregate metrics for robust statistical comparison. Chan et al. [3] introduce reliability metrics that go beyond mean return, and Colas et al. [4] provide guidelines for statistical comparisons.

Robust RL studies performance under distributional shift and adversarial perturbations, often with modified training objectives (e.g., minimax, CVaR) [5]. Distributional RL [6] models the return distribution rather than only its expectation. Return landscape analysis [7] highlights fragility modes invisible from

average return. Atari benchmarks [8] and procgen [9] stress generalization across environment variants.

ARCUS-H differs from all of the above: it does not modify the training objective, does not require model access, and focuses on behavioral *stability channels* rather than return distribution or policy robustness in the adversarial training sense. It standardizes a stress evaluation protocol and provides calibrated per-run diagnostics.

# 3 ARCUS-H Framework

## 3.1 MDP and notation

We consider a finite-horizon MDP $\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, r, \gamma, T)$ and a deterministic or stochastic policy $\pi_\theta$. An evaluation episode produces a trajectory $\tau = (s_1, a_1, \ldots, s_T, a_T)$ with return $G(\tau) = \sum_{t=1}^{T} \gamma^{t-1} r(s_t, a_t)$.

## 3.2 Behavioral stability channels

ARCUS-H constructs a per-episode stability score from five interpretable channels. Each channel is a scalar in $[0, 1]$ computed from episode statistics; none requires internal model access.

**Competence.** Let $R_e$ be the episode return and $\bar{R}_e$ an exponential moving average with smoothing $\alpha_c$. Competence measures improvement relative to the recent trend:

$$c_e = \sigma\left(6 \cdot \frac{R_e - \bar{R}_{e-1}}{S_R}\right), \quad \sigma(x) = \frac{1}{1 + e^{-x}}.$$

**Coherence.** For discrete actions, coherence is $h_e = 1 - \rho_e$ where $\rho_e$ is the within-episode action switch rate. For continuous actions:

$$h_e = \exp\left(-2 \cdot \frac{1}{T_e - 2} \sum_{t=3}^{T_e} \|j_{e,t}\|_2\right),$$

where $j_{e,t} = (a_{e,t} - a_{e,t-1}) - (a_{e,t-1} - a_{e,t-2})$ is the action jerk.

**Continuity.** A behavior signature vector $s_e$ summarizes episode action and reward statistics. Continuity measures local self-consistency across consecutive episodes:

$$t_e = \begin{cases} 1 & e = 1, \\ \exp(-\|s_e - s_{e-1}\|_2 / \sigma_c) & e > 1. \end{cases}$$

**Integrity.** During the pre-phase, an anchor signature $a$ is maintained as a running mean over pre-phase episode signatures. Integrity measures fidelity to this pre-phase behavioral anchor:

$$i_e = \begin{cases} 1 & \text{anchor undefined}, \\ \exp(-\|s_e - a\|_2 / \sigma_i) & \text{otherwise}. \end{cases}$$

**Meaning.** Let $V_e$ be the total constraint-violation mass and $G_e$ the regret sum over the episode. Meaning measures constraint respect and regret-free behavior:

$$m_e = \exp\left(-\alpha_m \cdot \frac{\max(0, V_e)}{T_e}\right) \cdot \exp\left(-\frac{\max(0, G_e)}{S_g}\right).$$

In stress-free baseline episodes, $V_e = G_e = 0$ so $m_e = 1$ structurally.

**Composite stability score.** Weights $w_c, w_h, w_t, w_i, w_m$ (summing to 1) combine channels into the per-episode stability score:

$$I_e = w_c c_e + w_h h_e + w_t t_e + w_i i_e + w_m m_e \in [0, 1].$$

Weights are inferred from baseline-phase channel variances: channels with higher baseline MAD receive lower weight, reflecting that noisy channels provide less reliable signal.

## 3.3 Phase schedule

Each evaluation run uses three contiguous phases: **pre** (episodes 1–$n_{pre}$), **shock** (episodes $n_{pre} + 1$–$n_{pre} + n_{shock}$), and **post** (remaining). With default episode count 120, each phase receives 40 episodes. Stress transformations apply exclusively during the shock phase.

## 3.4 Collapse scoring and adaptive calibration

**Direct robust z-scoring.** From the pre-phase episodes, ARCUS-H computes the median and MAD of each channel. During scoring, channels are z-scored relative to these pre-phase statistics:

$$z_i = \frac{\mu_i^{pre} - i_e}{1.4826 \cdot \mathrm{MAD}_i^{pre}},$$
$$z_d = \frac{d_e^{id} - \mu_d^{pre}}{1.4826 \cdot \mathrm{MAD}_d^{pre}},$$

where $d_e^{id} = \max(0, \bar{I}_{pre} - I_e)$ is the identity drop. This replaces a deficit-based approach that produces $\mathrm{MAD} = 0$ by construction (since $p20$ of the baseline ensures $80\%$ of deficits are zero), rendering z-scores undefined.

**Collapse score.** Channels are combined via adaptive weights derived from their MADs and passed through sigmoid sharpening with center $\gamma$:

$$S_e = \sigma(\beta \cdot (\mathrm{raw}_e - \gamma)), \quad \mathrm{raw}_e = \frac{w_m \sigma(z_m) + w_i \sigma(z_i) + w_d \sigma(z_d)}{w_m + w_i + w_d}.$$

**Adaptive threshold.** The binary collapse event threshold is set to the 95th percentile of $S_e$ values computed over the pre-phase:

$$\eta = \mathrm{p95}(\{S_e : e \in \mathcal{T}_{pre}\}).$$

This yields FPR $\approx \alpha = 0.05$ by construction without any free parameter.

**Baseline re-calibration.** Because the reference pass and the main evaluation pass use independent environment instances (necessary for concept drift calibration), their random reset sequences can diverge. ARCUS-H re-derives $\bar{I}_{pre}$, the robust channel statistics, and $\eta$ directly from the actual pre-phase episodes of each evaluation run, eliminating the calibration mismatch that would otherwise inflate FPR to 20–50%.

### 3.5 Metrics

**Shock collapse rate:** $CR_{shock} = \frac{1}{|\mathcal{T}_{shock}|} \sum_{e \in \mathcal{T}_{shock}} \mathbf{1}[S_e \geq \eta]$.

**Pre-to-shock drop:** $\Delta I = \mu_{pre}(I) - \mu_{shock}(I)$.

**Leaderboard score:**

$$\text{robust} = 0.55 \cdot \bar{I} + 0.30 \cdot (1 - CR_{shock}) + 0.15 \cdot \text{rwd_norm},$$

where $\text{rwd_norm} \in [0, 1]$ is per-environment min-max normalized return. Stability accounts for 85% of the dynamic component; reward contributes 15%.

## 4 Stress Schedules

ARCUS-H applies four stress schedules during the shock phase. If the agent outputs $a_t \sim \pi_\theta(\cdot|s_t)$, the harness executes a transformed action or reward:

$$a_t^{exec} = \begin{cases} a_t & t \in \mathcal{T}_{pre} \cup \mathcal{T}_{post}, \\ g(a_t; \phi) & t \in \mathcal{T}_{shock}. \end{cases} \quad (1)$$

### 4.1 Concept drift (CD)

The observation space is perturbed by an additive drift vector $\delta_t$ that evolves over the shock phase:

$$s_t^{exec} = s_t + \delta_t, \quad \delta_t = \delta_{t-1} + \epsilon_t, \quad \epsilon_t \sim \mathcal{N}(0, \sigma_{obs}^2 I).$$

The drift scale $\sigma_{obs}$ is auto-calibrated from the standard deviation of observations collected during the reference pass, ensuring the perturbation is commensurate with natural observation variability.

### 4.2 Resource constraint (RC)

Models reduced control authority. For continuous actions: $a_t^{exec} = \kappa a_t, \kappa \in (0, 1]$. For discrete actions: $a_t^{exec} = a_{default}$ with probability $p$, else $a_t$.

### 4.3 Trust violation (TV)

Models action-execution mismatch. For discrete actions: $a_t^{exec} = \pi_f(a_t)$ where $\pi_f$ is a fixed non-identity permutation. For continuous actions: $a_t^{exec} = \mathbf{M} a_t + \varepsilon_t$ where $\mathbf{M}$ is a fixed diagonal distortion matrix and $\varepsilon_t \sim \mathcal{N}(0, \sigma^2 I)$.

### 4.4 Valence inversion (VI)

Corrupts reward feedback: $r_t^{exec} = -r(s_t, a_t^{exec})$. This is the severest stressor by design—reward sign inversion renders the agent's optimization objective inconsistent with its learned policy.

## 5 Experimental Protocol

**Environments.** We evaluate 9 environments across three suites: **Classic control** (6 envs): `Acrobot-v1`, `CartPole-v1`, `FrozenLake-v1`, `MountainCar-v0`, `MountainCarContinuous-v0`, `Pendulum-v1`; **MuJoCo continuous control** (2 envs): `HalfCheetah-v4`, `Hopper-v4`; **Image-based** (1 env): `ALE/Pong-v5` (pixel observations, CNN policy).

**Algorithms.** We evaluate 7 algorithms: on-policy (PPO, A2C, TRPO) and off-policy (DQN, DDPG, SAC, TD3), implemented via Stable-Baselines3 [10] and SB3-Contrib. Not all algorithms are applicable to all environments (e.g., DQN requires discrete actions).

**Training.** Classic control: $3 \times 10^5$ steps. MuJoCo: $3 \times 10^5$ steps. Atari (Pong): $3 \times 10^6$ steps with frame stacking (4 frames, $84 \times 84$ grayscale).

**Evaluation.** Each configuration (algorithm $\times$ environment $\times$ eval mode $\times$ schedule) is evaluated across seeds 0–9 (10 seeds). Each run uses 120 episodes split evenly into pre/shock/post (40 each). We evaluate both `deterministic` (mean action) and `stochastic` (sampled action) modes.

**Calibration.** The adaptive threshold $\eta$ is derived from the actual pre-phase of each run. For concept drift, $\sigma_{obs}$ is calibrated from the reference pass. No parameters are tuned per environment.

## 6 Results

### 6.1 Scoring validation

Figure 1 shows the baseline FPR distribution across all 83 evaluated (env, algo, eval mode) combinations. The mean FPR is 2.0% (target $\alpha = 0.05$), with all but one run below 10%. The right panel confirms that stressor shock rates substantially exceed pre-phase rates, validating discriminative power. The adaptive p95 threshold achieves this calibration with no environment-specific tuning.

### 6.2 Overview: collapse rate across environments

Figure 2 provides a global view. Several patterns are immediately visible. First, valence inversion (VI) is the most damaging

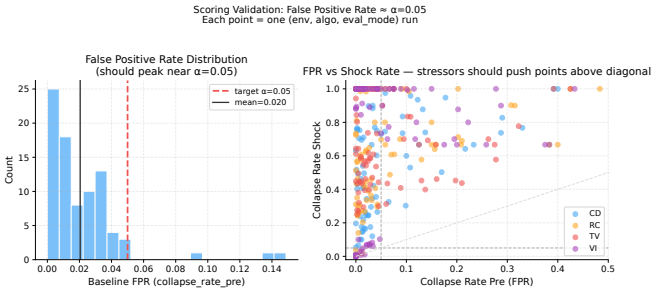


Figure 1: FPR validation. Left: histogram of baseline pre-phase collapse rates across all (env, algo, eval mode) runs; mean $= 2.0\%$, target $\alpha = 0.05$. Right: pre vs shock collapse rate per run; all stressor points lie above the diagonal.

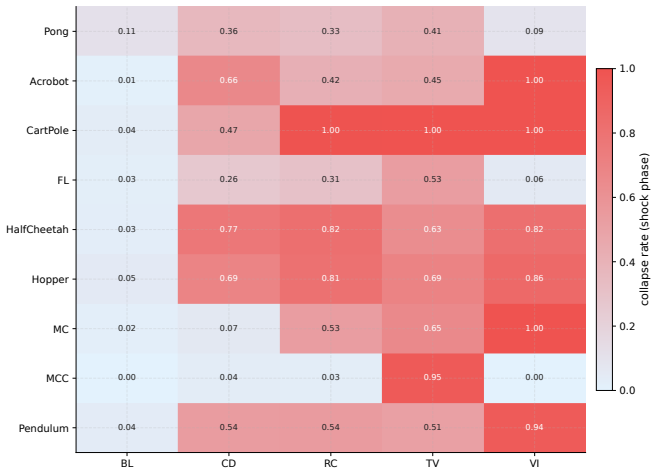


Figure 2: Mean collapse rate (shock phase) per environment and stressor, averaged across algorithms, seeds, and eval modes. BL = baseline (FPR). White cells indicate combinations not applicable to that environment.

stressor for most environments, consistent with its design as a reward-sign inversion. Three environments are notable exceptions, each with a mechanistic explanation: **FrozenLake** (TV $>$ VI, because sparse rewards provide VI with little grip on a near-random agent); **MountainCarContinuous** (TV $\gg$ VI, because continuous action distortion destroys the precise force profile required to escape the valley); and **Pong** (TV $>$ VI $\approx$ BL, because reward inversion allows a competent Pong agent to form a coherent counter-strategy of deliberately missing, maintaining high stability despite inverted reward). These are findings, not failures.

## 6.3 Reward does not predict stability

Figure 3 addresses the central question of ARCUS-H: does reward performance predict behavioral stability under stress? The answer is clearly negative. Pearson $r = +0.14$, $p = 0.364$ between normalized reward and VI collapse rate (Figure 3, right panel). The null correlation holds across both raw and normalized reward scales. High-reward agents (e.g., SAC on HalfCheetah) collapse at rates of $83\%$ under multiple stressors, while lower-reward agents (e.g., DQN on MountainCar) show

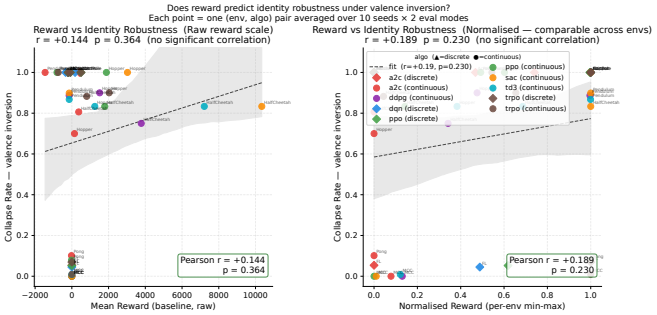


Figure 3: Reward vs. stability. Each point is one (env, algo) pair averaged over 10 seeds and 2 eval modes. Left: raw reward scale. Right: per-environment normalized reward. Pearson $r = +0.14$, $p = 0.364$ (no significant correlation in either panel).

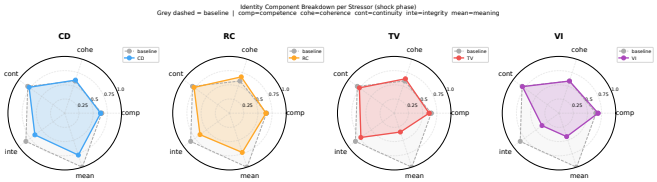


Figure 4: Stability channel breakdown per stressor (shock phase). Grey dashed = baseline. CD attacks integrity; TV attacks all channels uniformly; VI attacks meaning; RC reduces competence and coherence.

near-zero collapse. This divergence is the primary motivation for ARCUS-H as a complementary evaluation axis.

## 6.4 Stressor signatures across stability channels

Figure 4 shows which stability channels each stressor primarily attacks (shock phase means vs. baseline, averaged across all envs/algos). Each stressor leaves a distinct fingerprint. Concept drift depresses integrity most severely, consistent with its mechanism of shifting the observation distribution away from the pre-phase anchor. Trust violation suppresses all channels comparably, reflecting the global disruption of action-execution correspondence. Valence inversion attacks meaning (constraint-respect signal) most strongly, which is structurally expected since reward inversion generates positive feedback for constraint-violating behaviors. Resource constraint primarily reduces competence and coherence, consistent with reduced action resolution.

## 6.5 Evaluation mode robustness

Figure 5 shows the correlation between deterministic and stochastic evaluation mode verdicts across all (env, algo) pairs. Pearson $r = 0.82$ (CD), $0.81$ (RC), $0.65$ (TV), $0.96$ (VI). The weaker correlation for TV reflects environments where stochastic policy entropy partially mitigates action-permutation effects; for all other stressors, eval mode choice has minimal impact on ARCUS-H rankings.

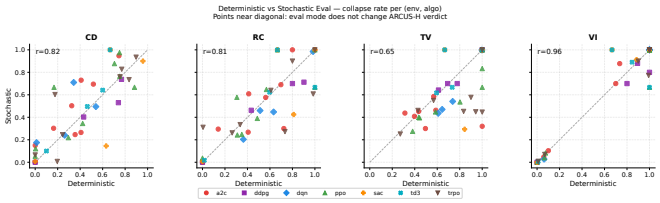


Figure 5: Deterministic vs. stochastic eval mode collapse rates per (env, algo). Points near the diagonal indicate that eval mode does not change ARCUS-H verdict. Pearson $r$ is annotated per stressor.

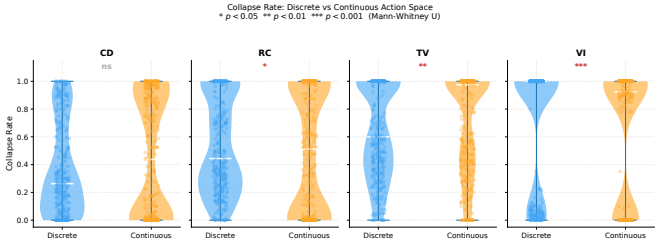


Figure 6: Collapse rate: discrete vs. continuous action spaces per stressor. Significance: $*p<0.05$, $**p<0.01$, $***p<0.001$ (Mann-Whitney U). CD shows no significant difference; RC, TV, VI all favor continuous being more vulnerable.

## 6.6 Action space and policy family effects

Figure 6 shows that continuous action spaces are significantly more vulnerable than discrete under RC, TV, and VI (Mann-Whitney $p < 0.05$, $p < 0.01$, $p < 0.001$ respectively), while CD produces no significant difference. This is consistent with the stressor mechanisms: action-space distortions (RC, TV) are more damaging when the action space is high-dimensional and the precise magnitude of actions is critical for task performance (e.g., MuJoCo locomotion).

Figure 7 shows that on-policy (PPO, A2C, TRPO) and off-policy (DQN, DDPG, SAC, TD3) families exhibit comparable collapse rates across stressors, confirming that ARCUS-H captures a property of the agent-environment interaction that is not strongly mediated by the learning algorithm family.

## 6.7 Suite-level depth: MuJoCo vs Classic vs Image

Figure 8 compares collapse rates across the three environment suites. MuJoCo agents collapse at 73–84% under all stressors, substantially higher than Classic control (33–69%) and Image/Pong (9–41%). Crucially, this ordering is *inverted* relative to reward: MuJoCo agents achieve the highest absolute rewards yet show the greatest stress fragility. This is consistent with the reward/stability divergence finding (Section 6.3) and reflects that high-performance locomotion policies exploit precise continuous action dynamics that collapse under even mild perturbation.

## 6.8 Statistical reliability

Figure 9 shows per-seed consistency. Coefficient of variation across 10 seeds is CV= 0.15 (CD), 0.09 (RC), 0.06 (TV), 0.04

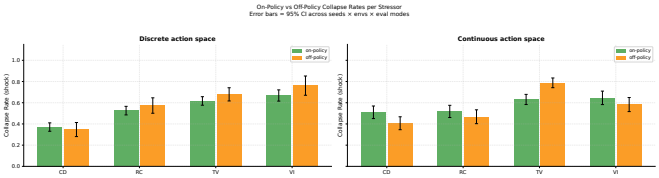


Figure 7: On-policy vs. off-policy collapse rates per stressor and action space type. Error bars: 95% CI across seeds × envs × eval modes. Both families collapse comparably, confirming ARCUS-H is algorithm-family-agnostic.

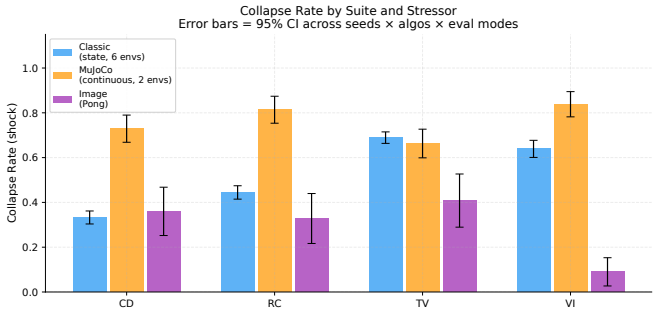


Figure 8: Collapse rate by environment suite and stressor (95% CI). MuJoCo agents collapse most severely despite achieving the highest reward, demonstrating the reward/stability divergence at the suite level.

(VI). Rankings are stable: no single seed drives conclusions. Table 1 (Appendix) reports collapse rates per environment and stressor; Table 3 (Appendix) reports per-environment FPR.

## 6.9 Algorithm vulnerability profiles

Table 2 (Appendix) reports the most damaging middle stressor (CD/RC/TV, excluding VI) per algorithm-environment pair. Several patterns emerge. Trust violation is the most common worst-case stressor for discrete-action environments, whereas resource constraint is dominant for MuJoCo continuous control. TRPO shows a strong susceptibility to concept drift across multiple environments, consistent with its conservative policy update that may be more sensitive to distribution shift. DQN (off-policy, replay buffer) and PPO (on-policy, clipped objective) show different vulnerability profiles, confirming that ARCUS-H discriminates across algorithm families.

# 7 Discussion

ARCUS-H provides calibrated, reproducible stress diagnostics that are complementary to return. The central empirical finding—Pearson $r = +0.14$ between reward and collapse rate—confirms that current RL benchmarks systematically overlook a dimension of agent behavior that becomes critical when evaluation conditions deviate from training. The reward/stability divergence is not merely a statistical artifact: it is structurally reproducible across 9 environments and 7 algorithms, and is most pronounced precisely in the environments where agents achieve the strongest task performance (MuJoCo).

The stressor-channel signatures (Figure 4) demonstrate that

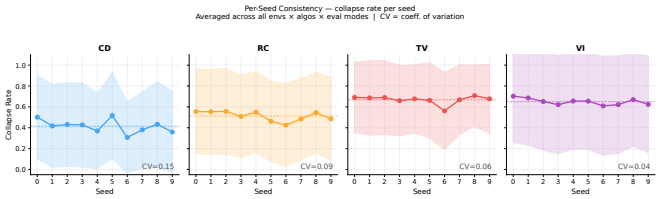


Figure 9: Collapse rate per seed (mean across all envs $\times$ algos $\times$ eval modes). CV values confirm results are not driven by individual seeds.

the five stability channels are not redundant. Each captures a different facet of behavioral collapse, providing diagnostic resolution beyond a single composite score. This per-channel interpretability is a deliberate design property: ARCUS-H is intended as a diagnostic tool, not a black-box rating.

The Pong valence-inversion finding is worth highlighting: a competent Pong agent subjected to inverted reward maintains high stability scores because it can coherently adopt a new strategy (deliberately losing). This shows that ARCUS-H measures *behavioral coherence*, not reward-tracking fidelity—an agent that coherently pursues a wrong objective still scores high on stability.

# 8 Limitations

**Stressor coverage.** Current stressors cover action and reward perturbations. Observation corruption, latent dynamics shift, and partial observability are not yet included, though all are relevant in deployment scenarios.

**Stationarity assumption.** ARCUS-H assumes the agent's behavioral distribution is stationary across the pre/shock/post split before any stressor is applied. Procedurally-generated environments (e.g., Procgen CoinRun) violate this assumption: each episode draws a new level, causing the pre-phase score distribution to differ from the shock-phase distribution even without a stressor. We evaluated CoinRun A2C with stochastic evaluation mode (which partially mitigates level-difficulty variance) but exclude it from main results. Level-conditioned calibration is left to future work.

**Image-based off-policy.** DQN on Atari requires approximately $10\times$ more training steps than on-policy methods to reach competence, making matched-performance comparison infeasible within our compute budget. Image-based off-policy evaluation is left to future work.

**Sensitivity to channel construction.** The five stability channels use specific functional forms and scale parameters. While rankings were qualitatively stable under $\pm 20\%$ weight perturbations, the sensitivity of conclusions to alternative channel definitions (e.g., different jerk norms, different signature vector construction) has not been fully characterized.

# 9 Future Work

Immediate next steps include: (1) observation-space stressors (sensor noise, frame corruption for image-based policies); (2) level-conditioned calibration for procedural environments; (3) long-horizon evaluation with gradual drift rather than step-change shocks; (4) integration of ARCUS-H into standard training pipelines to study the relationship between training robustness and evaluation stability.

# 10 Code Availability

The ARCUS-H harness and all scripts used to generate the reported results are available at:

`https://github.com/karimzn00/ARCUSH`

# 11 Additional Tables and Figures

Table 1: Mean collapse rate (shock phase) per environment and stressor, averaged across algorithms, seeds, and evaluation modes. Higher = greater identity collapse under stress. **Bold** = worst stressor per row (excluding BL).

| Environment | BL | CD | RC | TV | VI |
|---|---|---|---|---|---|
| Pong | 0.11 | 0.36 | 0.33 | **0.41** | 0.09 |
| Acrobot | 0.01 | 0.66 | 0.42 | 0.45 | **1.00** |
| CartPole | 0.04 | 0.47 | 1.00 | **1.00** | **1.00** |
| FL | 0.03 | 0.26 | 0.31 | **0.53** | 0.06 |
| HalfCheetah | 0.03 | 0.77 | **0.82** | 0.63 | 0.82 |
| Hopper | 0.05 | 0.69 | 0.81 | 0.69 | **0.86** |
| MC | 0.02 | 0.07 | 0.53 | 0.65 | **1.00** |
| MCC | 0.00 | 0.04 | 0.03 | **0.95** | 0.00 |
| Pendulum | 0.04 | 0.54 | 0.54 | 0.51 | **0.94** |

Table 2: Most damaging stressor (excluding VI) per algorithm and environment. CD = concept drift, RC = resource constraint, TV = trust violation. Blank cells indicate the algorithm was not evaluated on that environment.

| Algo | Pong | Acrobot | CartPole | FL | HalfCheetah | Hopper | MC | MCC | Pendulum |
|---|---|---|---|---|---|---|---|---|---|
| a2c | CD | CD | TV | TV | RC | RC | TV | TV | RC |
| ddpg | – | – | – | – | RC | RC | – | TV | TV |
| dqn | – | CD | TV | TV | – | – | TV | – | – |
| ppo | RC | CD | TV | TV | TV | CD | TV | TV | CD |
| sac | – | – | – | – | TV | TV | – | TV | RC |
| td3 | – | – | – | – | TV | RC | – | TV | TV |
| trpo | – | CD | TV | TV | RC | CD | TV | CD | CD |

Table 3: Baseline false positive rate (FPR) per environment, averaged across algorithms, seeds, and evaluation modes. Target $\alpha = 0.05$. † denotes FPR $> 0.10$ (see §**??**).

| Environment | FPR mean | FPR std |
|---|---|---|
| Pong | 0.034 | 0.023 |
| Acrobot | 0.010 | 0.024 |
| CartPole | 0.025 | 0.039 |
| FL | 0.027 | 0.032 |
| HalfCheetah | 0.011 | 0.020 |
| Hopper | 0.027 | 0.022 |
| MC | 0.010 | 0.021 |
| MCC | 0.001 | 0.011 |
| Pendulum | 0.039 | 0.069 |
| Overall | 0.020 | 0.029 |

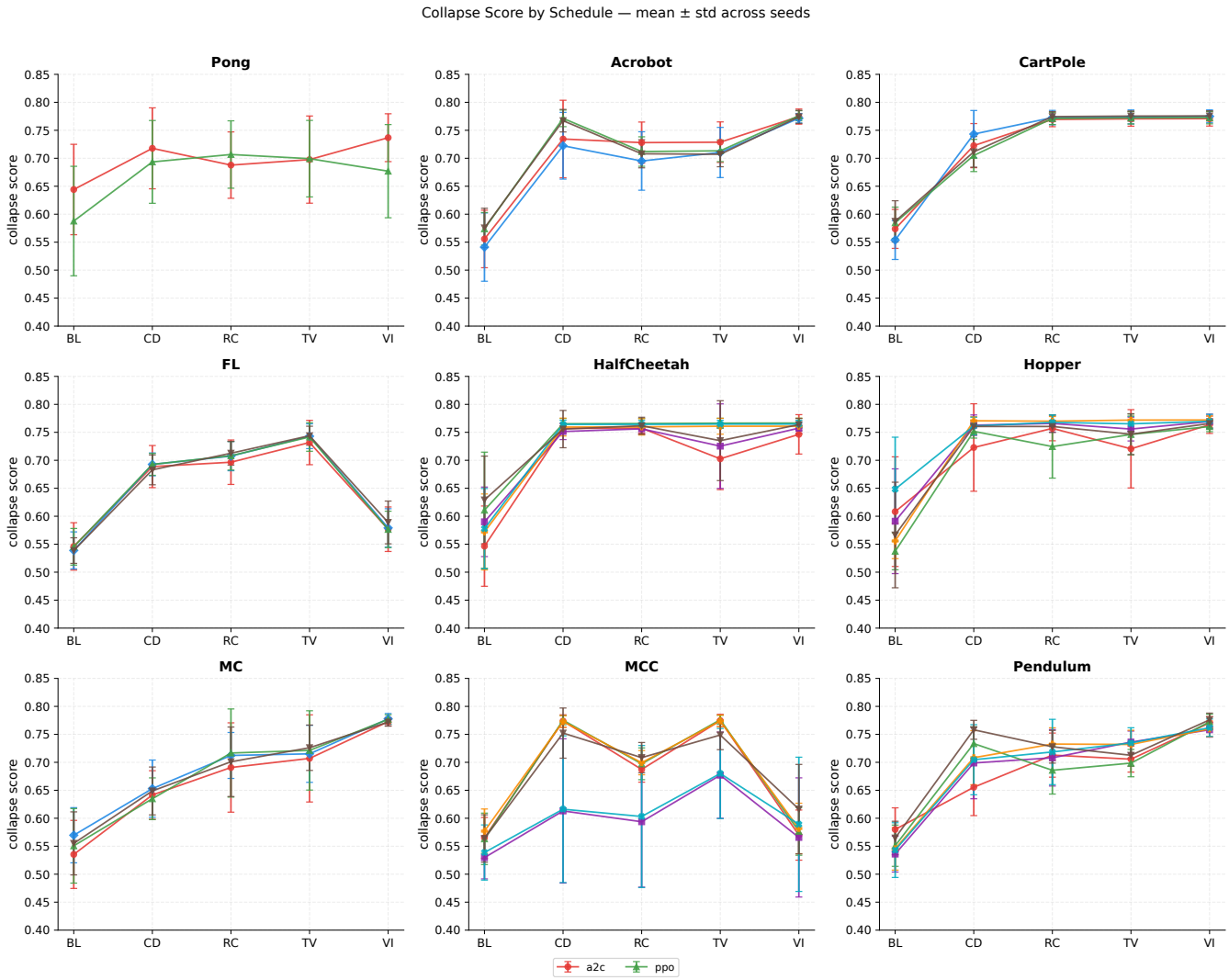


Figure 10: Collapse score by schedule per environment. Each line is one algorithm; error bars = std across 10 seeds.

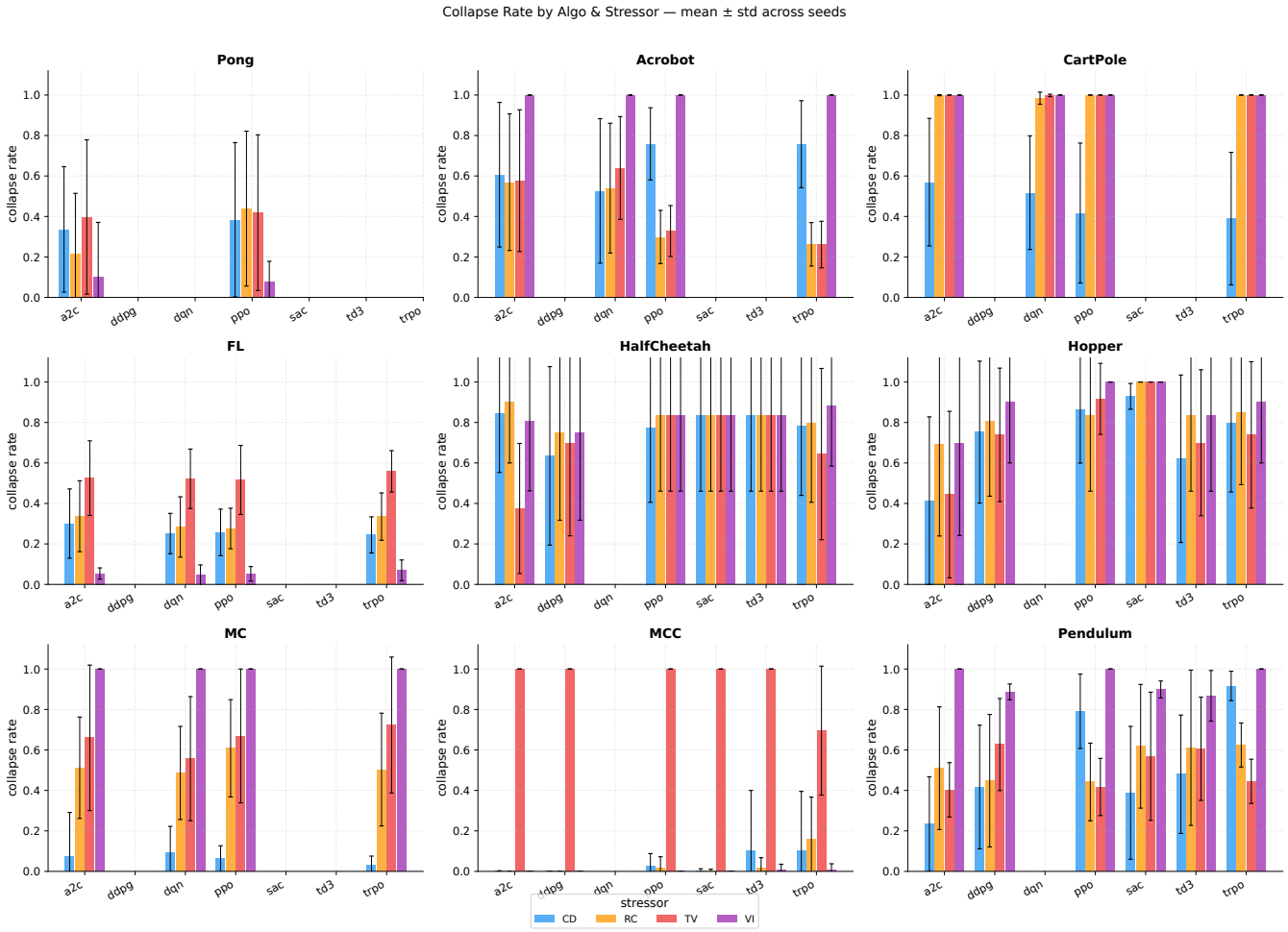


Figure 11: Collapse rate by algorithm and stressor per environment. Bars = mean across seeds; error bars = std.

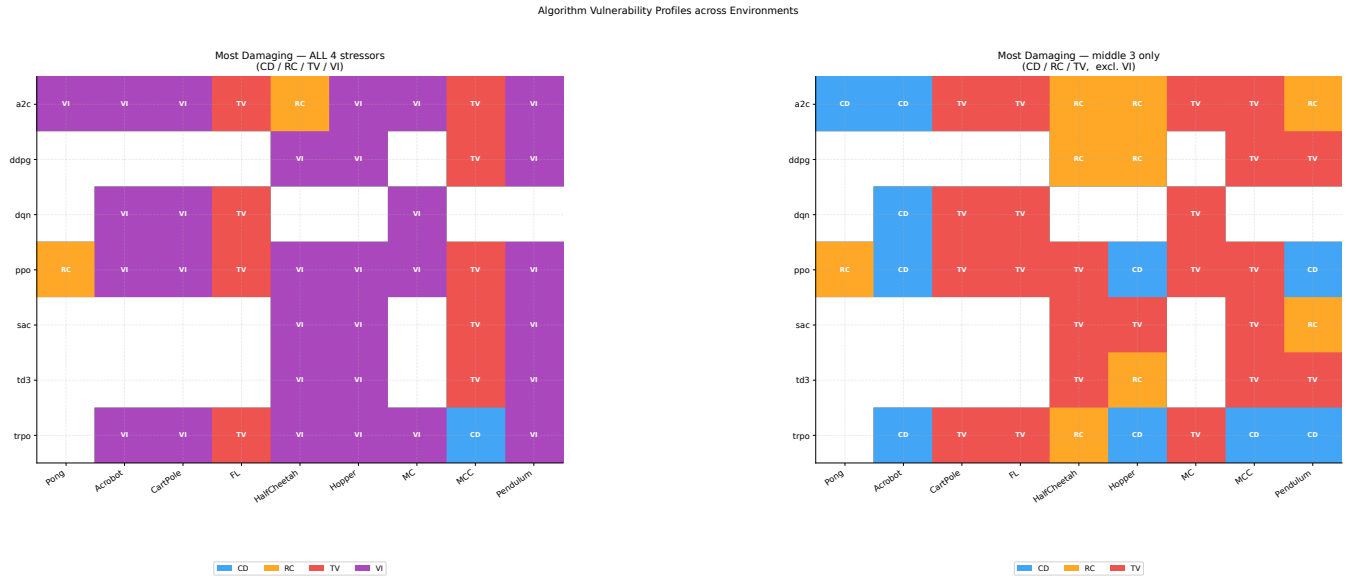


Figure 12: Algorithm vulnerability profiles. Left: most damaging stressor among all 4 (CD/RC/TV/VI). Right: most damaging among middle 3 only (CD/RC/TV, excluding VI) to reveal finer-grained algo-specific signatures.

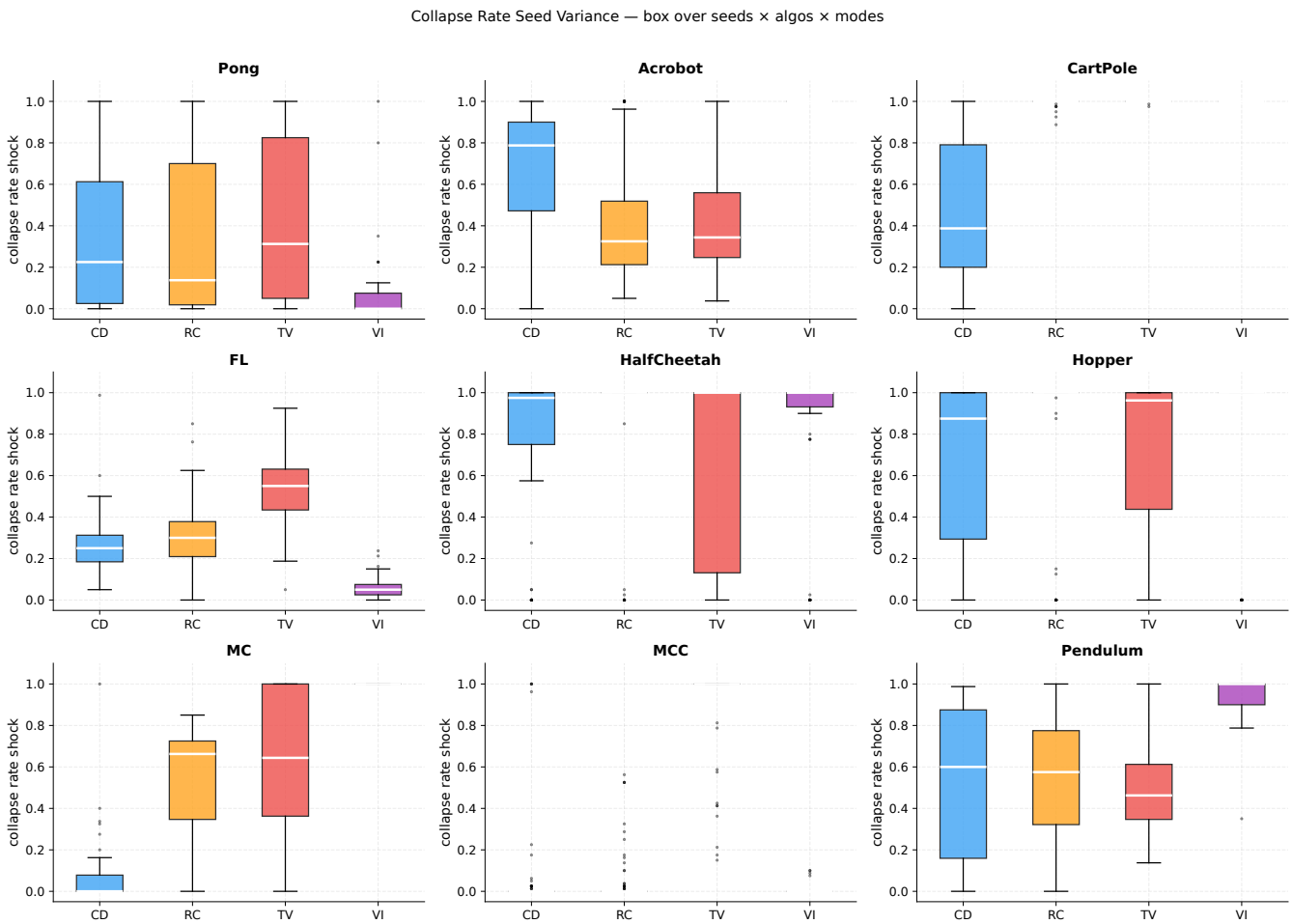


Figure 13: Distribution of collapse rate (shock) across seeds × algos × eval modes per environment and stressor. Box = IQR; whiskers = 1.5×IQR.

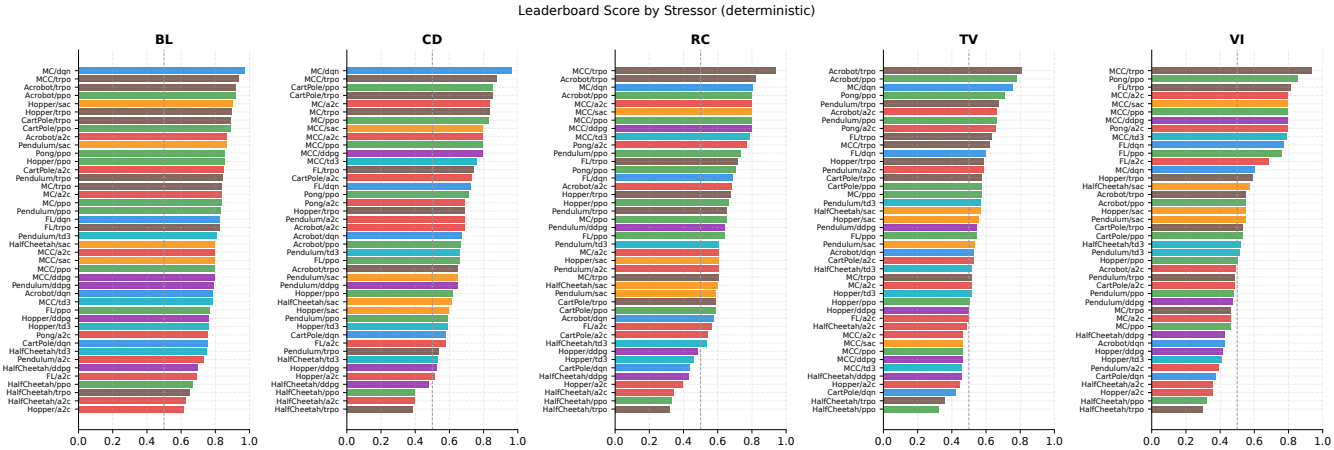


Figure 14: Leaderboard scores by stressor (deterministic eval). Higher = more robust. Dashed line at 0.5.

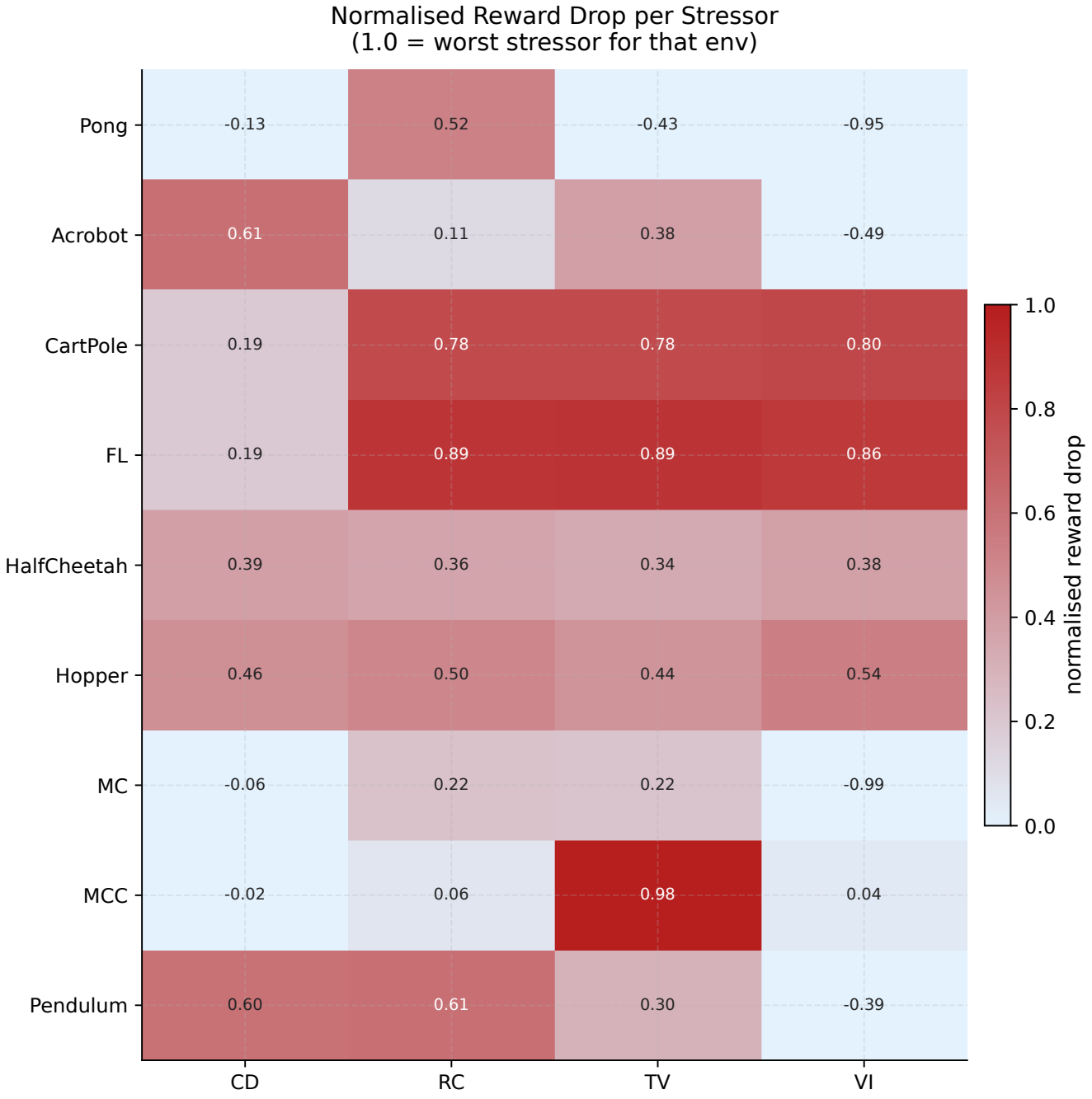


Figure 15: Normalised reward drop per stressor and environment (1.0 = worst stressor for that environment).

# References


[1] P. Henderson et al. Deep Reinforcement Learning that Matters. *AAAI*, 2018.

[2] R. Agarwal et al. Deep Reinforcement Learning at the Edge of the Statistical Precipice. *NeurIPS*, 2021.

[3] S. C. Y. Chan et al. Measuring the Reliability of Reinforcement Learning Algorithms. *ICLR*, 2019.

[4] C. Colas et al. A Hitchhiker's Guide to Statistical Comparisons of Reinforcement Learning Algorithms. *ICLR Reproducibility Workshop*, 2019.

[5] R. T. Rockafellar and S. Uryasev. Optimization of Conditional Value-at-Risk. *Journal of Risk*, 2:21–42, 2000.

[6] M. G. Bellemare, W. Dabney, and M. Rowland. Distributional Reinforcement Learning. *Foundations and Trends in Machine Learning*, 2022.

[7] N. Rahn et al. Policy Optimization in a Noisy Neighborhood: On Return Landscapes in Continuous Control. *NeurIPS*, 2023.

[8] M. G. Bellemare, Y. Naddaf, J. Veness, and M. Bowling. The Arcade Learning Environment: An Evaluation Platform for General Agents. *JAIR*, 47:253–279, 2013.

[9] K. Cobbe et al. Leveraging Procedural Generation to Benchmark Reinforcement Learning. *ICML*, 2020.

[10] A. Raffin et al. Stable-Baselines3: Reliable Reinforcement Learning Implementations. *JMLR*, 22(268):1–8, 2021.